TrashTracker

Systeme IoT de Surveillance des Poubelles Publiques

Realise par :

Benazza mohammed
Labrazi mohammed yassine
Achalhi abdeljalil
Badr boulaid

Module :	Systemes Embarques & IoT
Plateforme :	ESP32 (Wokwi) + Python (OpenCV)
Protocole :	MQTT + HTTP
Annee :	2025–2026

Table des matières

1 Introduction **2**
1.1 Contexte et Problematique . . . 2
1.2 Objectifs du Projet . . . 2

2 Architecture du Systeme **2**
2.1 Vue d'ensemble . . . 2
2.2 Composants Materiels . . . 3
2.3 Topics MQTT . . . 3
2.4 Etude comparative : MQTT vs HTTP . . . 3

3 Fonctionnement Detaille **3**
3.1 Module de Vision — Script Python (OpenCV) . . . 3
3.1.1 Detection de l'etat de remplissage . . . 4
3.1.2 Detection de mouvement (intrusion animale) . . . 4
3.1.3 Visualisation en temps reel . . . 4
3.2 Module de Signalisation — Firmware ESP32 (C++) . . . 4

4 Code Source **4**
4.1 Firmware ESP32 — `main.cpp` . . . 4
4.2 Script de Vision — `camera.py` . . . 5

5 Simulation Wokwi **7**
5.1 Description du Circuit . . . 7
5.2 Reseau et Connectivite . . . 8

6 Resultats et Tests **8**
6.1 Scenarios Testes . . . 9
6.2 Analyse des Performances . . . 9

7 Limites et Perspectives d'Amelioration **9**
7.1 Limites Actuelles . . . 9
7.2 Perspectives d'Amelioration . . . 9

8 Conclusion **10**

Annexes **11**

1. Introduction

1.1. Contexte et Problematique

La gestion des dechets urbains represente un defi majeur pour les collectivites. Les poubelles publiques debordent souvent faute d'une collecte optimisee, entrainant des nuisances visuelles, olfactives et sanitaires. De plus, les animaux errants (notamment les chats) fouillent regulierement les bacs, dispersant les dechets sur la voie publique.

Problemes identifies

- Poubelles pleines non detectees a temps, causant des debordements.
- Absence d'indicateur visuel de l'etat de remplissage pour les agents de collecte.
- Intrusions d'animaux (chats, rongeurs) dans les bacs publics non signalees.
- Tournees de collecte non optimisees, couteuses et peu ecologiques.

1.2. Objectifs du Projet

Le projet **TrashTracker** propose une solution IoT bas cout pour :

- Detecter en temps reel si une poubelle est **pleine** ou **disponible** via analyse video.
- Signaler l'etat par un indicateur lumineux : **LED verte** = disponible, **LED rouge** = plein.
- **Declencher une alarme sonore** (buzzer) des qu'un mouvement est detecte pres du bac.
- Communiquer les donnees via le protocole **MQTT** pour une supervision centralisee.

2. Architecture du Systeme

2.1. Vue d'ensemble

Le systeme TrashTracker s'articule autour de deux composants principaux communicant via un broker MQTT public.

Camera + Python	Broker MQTT	ESP32
Analyse video (OpenCV)	HiveMQ public	LEDs + Buzzer

2.2. Composants Materiels

Composant	Pin ESP32	Role
LED Verte	GPIO 18	Indique que la poubelle est disponible
LED Rouge	GPIO 19	Indique que la poubelle est pleine
Buzzer	GPIO 21	Alarme sonore lors de la detection de mouvement
ESP32 DevKit v1	—	Micro-controleur Wi-Fi, souscrit aux topics MQTT

Table 1 – Composants materiels du module de signalisation

2.3. Topics MQTT

Topic	Valeurs	Description
`city/bin/1/status`	`FULL` / `OK`	Etat de remplissage de la poubelle
`city/bin/1/alert`	`ANIMAL`	Alerte mouvement detecte

Table 2 – Topics MQTT utilises dans le projet

2.4. Etude comparative : MQTT vs HTTP

Dans la phase de conception, deux protocoles de communication ont ete etudies pour assurer la liaison entre le module de vision Python et l'unite ESP32. Bien que l'architecture HTTP soit une alternative courante, le protocole MQTT a ete prefere pour sa robustesse.

Critere	MQTT (Choisi)	HTTP (Etudie)
Modele	Publish / Subscribe (via Broker)	Request / Response (Direct)
Couplage	Faible (Decouplage total)	Fort (Le client doit connaitre l'IP)
Reseau	Independant de l'IP locale	Necessite une IP fixe ou statique
Légereté	Tres elevee (IoT optimal)	Faible (Headers lourds)
Reconnexion	Native et automatique	Manuelle et complexe a gerer

Table 3 – Comparaison des protocoles de communication IoT

Le choix de **MQTT** se justifie par le besoin de scalabilite du projet : un broker centralise permet de gerer des centaines de bacs sans avoir a configurer manuellement les adresses IP de chaque capteur et actionneur.

3. Fonctionnement Detaille

3.1. Module de Vision — Script Python (OpenCV)

Le script Python s'execute sur un ordinateur equipe d'une camera. Il capture en continu le flux video, analyse une region d'interet (ROI) correspondant a la zone de la poubelle, et publie l'etat sur le broker MQTT toutes les 2 secondes.

3.1.1. Detection de l'etat de remplissage

La luminosite moyenne de la ROI est calculee. Si elle est inferieure au seuil `FULL_BRIGHTNESS_THRESH` `= 100`, la poubelle est consideree comme pleine (zone sombre = bac obstrue par les dechets).

Luminosite moyenne	Etat publie	Signalisation
< 100	`FULL`	LED Rouge
≥ 100	`OK`	LED Verte

3.1.2. Detection de mouvement (intrusion animale)

La difference absolue entre deux frames consecutives dans la ROI est calculee. Si la moyenne de cette difference depasse le seuil `MOTION_THRESHOLD = 20`, un message `ANIMAL` est publie sur le topic d'alerte.

3.1.3. Visualisation en temps reel

Le script affiche un rectangle colore autour de la ROI (vert = OK, rouge = FULL), ainsi que la valeur de luminosite et le score de mouvement superposes sur le flux video.

3.2. Module de Signalisation — Firmware ESP32 (C++)

L'ESP32 se connecte au reseau Wi-Fi, puis au broker MQTT, et souscrit aux deux topics. En fonction des messages recus, il actionne les LEDs ou le buzzer.

Logique de controle

- **Topic `status` = "FULL"** ⇒ LED rouge allumee, LED verte eteinte.
- **Topic `status` = "OK"** ⇒ LED verte allumee, LED rouge eteinte.
- **Topic `alert` = "ANIMAL"** ⇒ Buzzer active pendant 2 secondes.

4. Code Source

4.1. Firmware ESP32 — `main.cpp`

```cpp
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid        = "Wokwi-GUEST";
const char* password    = "";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient   espClient;
PubSubClient client(espClient);

#define LED_GREEN 18
#define LED_RED   19
#define BUZZER    21

```

```
void callback(char* topic, byte* payload, unsigned int length) {
  String message;
  for (int i = 0; i < length; i++) {
    message += (char)payload[i];
  }
  String topicStr = String(topic);

  if (topicStr == "city/bin/1/status") {
    if (message == "FULL") {
      digitalWrite(LED_RED,   HIGH);
      digitalWrite(LED_GREEN, LOW);
    } else {
      digitalWrite(LED_GREEN, HIGH);
      digitalWrite(LED_RED,   LOW);
    }
  }

  if (topicStr == "city/bin/1/alert" && message == "ANIMAL") {
    digitalWrite(BUZZER, HIGH);
    delay(2000);
    digitalWrite(BUZZER, LOW);
  }
}

void reconnect() {
  while (!client.connected()) {
    if (client.connect("ESP32_BIN_1")) {
      client.subscribe("city/bin/1/status");
      client.subscribe("city/bin/1/alert");
    }
  }
}

void setup() {
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED,   OUTPUT);
  pinMode(BUZZER,    OUTPUT);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(100);

  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();
}
```

Listing 1 – Firmware ESP32 : Wi-Fi, MQTT et controle des actionneurs

4.2. Script de Vision — `camera.py`

```
import cv2
import time
import paho.mqtt.publish as publish
```

```python

BROKER       = "broker.hivemq.com"
TOPIC_STATUS = "city/bin/1/status"
TOPIC_ALERT  = "city/bin/1/alert"

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Erreur : impossible d'ouvrir la camera")
    exit()

BIN_X, BIN_Y, BIN_W, BIN_H   = 200, 150, 200, 200
FULL_BRIGHTNESS_THRESHOLD     = 100
MOTION_THRESHOLD              = 20
MQTT_INTERVAL                 = 2

last_mqtt_time = 0
ret, prev_frame = cap.read()
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    roi  = frame[BIN_Y:BIN_Y+BIN_H, BIN_X:BIN_X+BIN_W]
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

    brightness = gray.mean()
    status = "FULL" if brightness < FULL_BRIGHTNESS_THRESHOLD else "OK"

    prev_roi = prev_gray[BIN_Y:BIN_Y+BIN_H, BIN_X:BIN_X+BIN_W]
    diff     = cv2.absdiff(prev_roi, gray)
    motion   = diff.mean()
    animal_detected = motion > MOTION_THRESHOLD

    now = time.time()
    if now - last_mqtt_time > MQTT_INTERVAL:
        publish.single(TOPIC_STATUS, status, hostname=BROKER)
        if animal_detected:
            publish.single(TOPIC_ALERT, "ANIMAL", hostname=BROKER)
        last_mqtt_time = now

    color = (0, 0, 255) if status == "FULL" else (0, 255, 0)
    cv2.rectangle(frame, (BIN_X, BIN_Y),
                  (BIN_X+BIN_W, BIN_Y+BIN_H), color, 2)
    cv2.putText(frame, f"STATUS: {status}",
                (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    cv2.putText(frame, f"Brightness: {int(brightness)}",
                (20, 80), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,255,0), 2)
    cv2.putText(frame, f"Motion: {int(motion)}",
                (20, 110), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,255), 2)

    cv2.imshow("Smart Trash Bin Vision", frame)
    prev_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Listing 2 – Script Python : analyse video OpenCV et publication MQTT

5. Simulation Wokwi

5.1. Description du Circuit

La simulation a ete realisee sur la plateforme **Wokwi** (https://wokwi.com), qui permet de simuler des cartes ESP32 sans materiel physique. Le circuit comprend :

— Un **ESP32 DevKit v1** comme unite centrale.

— Une **LED verte** (GPIO 18) signalant que la poubelle est disponible.

— Une **LED rouge** (GPIO 19) signalant que la poubelle est pleine.

— Un **buzzer piezoelectrique** (GPIO 21) declenchant l'alarme anti-intrusion.

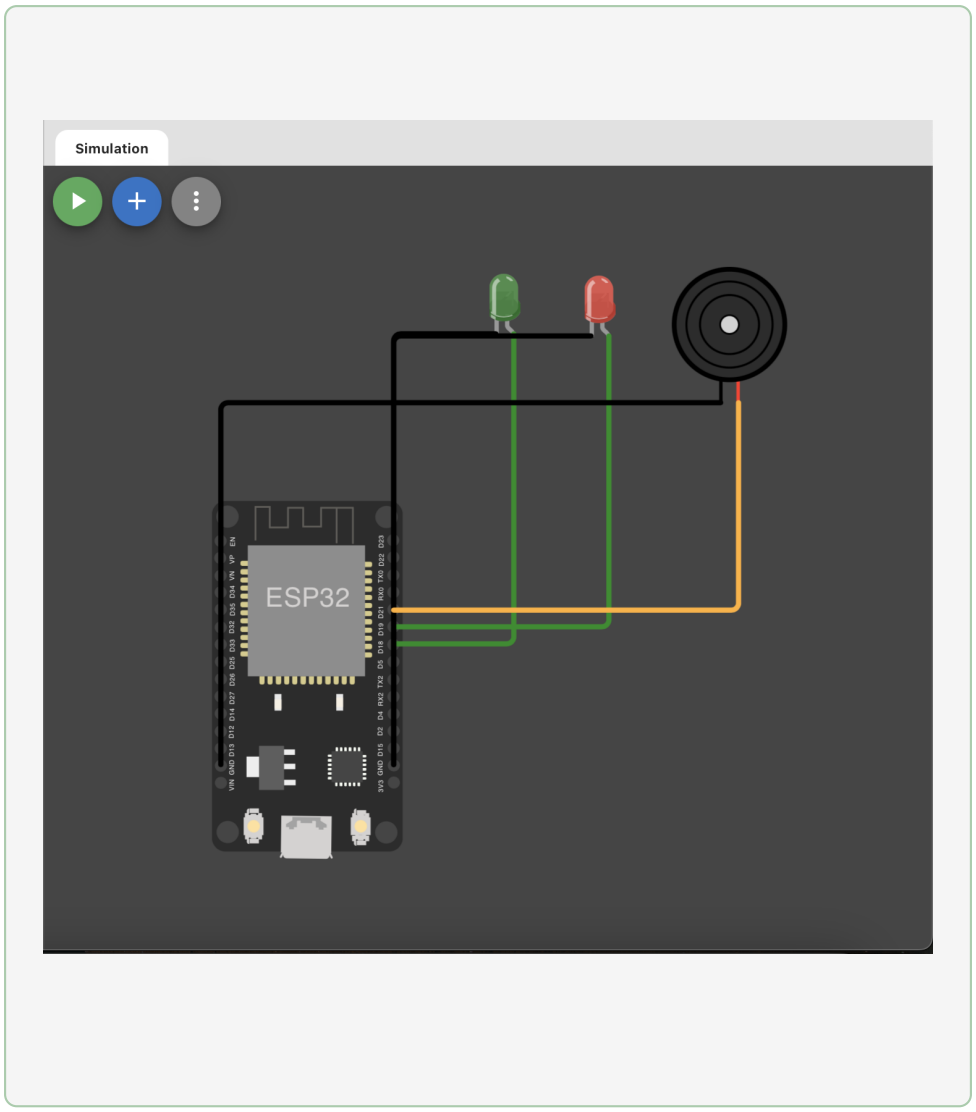

FIGURE 1 – Schema de cablage de la simulation Wokwi — TrashTracker

5.2. Reseau et Connectivite

Le reseau Wi-Fi **Wokwi-GUEST** est un reseau fictif integre a Wokwi permettant aux simulations d'acceder a internet (et donc au broker MQTT HiveMQ) sans configuration supplementaire.

6. Resultats et Tests

6.1. Scenarios Testes

Scenario	Message MQTT	Resultat attendu	Obtenu
Poubelle vide	`status = OK`	LED verte allumee	Conforme
Poubelle pleine	`status = FULL`	LED rouge allumee	Conforme
Mouvement detecte	`alert = ANIMAL`	Buzzer pendant 2s	Conforme
Deconnexion MQTT	—	Reconnexion automatique	Conforme

Table 4 – Tableau des scenarios de test

6.2. Analyse des Performances

- **Latence MQTT** : inferieure a 500 ms entre la detection Python et l'action sur l'ESP32.
- **Frequence de publication** : toutes les 2 secondes (configurable via `MQTT_INTERVAL`).
- **Fiabilite** : la reconnexion automatique garantit la continuite du service en cas de coupure reseau.
- **Seuils ajustables** : `FULL_BRIGHTNESS_THRESHOLD` et `MOTION_THRESHOLD` adaptables selon les conditions d'eclairage.

7. Limites et Perspectives d'Amelioration

7.1. Limites Actuelles

Points d'amelioration identifies

- **Detection par luminosite** : sensible aux variations d'eclairage exterieur. Un modele de machine learning serait plus robuste.
- **Broker MQTT public** : HiveMQ public n'offre aucune securite (pas d'authentification, pas de chiffrement TLS).
- **Un seul bac simule** : la scalabilite vers un parc de centaines de bacs necessite une architecture plus avancee.
- **Absence de persistance** : les donnees ne sont pas stockees, aucun historique n'est disponible.

7.2. Perspectives d'Amelioration

- **Detection par IA** : integrer un modele de classification (YOLO, MobileNet) pour une detection plus precise.
- **Tableau de bord** : developper une interface web (Node-RED, Grafana) pour visualiser l'etat de tous les bacs.
- **Base de donnees** : stocker l'historique dans InfluxDB ou MongoDB pour optimiser les tournees de collecte.
- **Securite** : migrer vers un broker MQTT avec authentification et chiffrement TLS/SSL.

— **Energie** : explorer l'alimentation solaire et le mode deep sleep de l'ESP32.

— **Capteur ultrasons** : ajouter un HC-SR04 pour mesurer le niveau sans dependance a la luminosite.

8. Conclusion

Le projet **TrashTracker** demontre qu'une solution IoT efficace et economique peut etre construite a partir de composants simples et de logiciels open source. La comparaison menee entre les protocoles MQTT et HTTP souligne l'importance du choix de l'architecture reseau : le modele « Publish/Subscribe » s'impose ici pour assurer le decouplage et la flexibilite necessaire a une Smart City.

La simulation realisee sur Wokwi valide le fonctionnement du systeme sur tous les scenarios testes. Ce prototype constitue une base solide pour un deploiement reel.

Points cles du projet

— Architecture IoT complete : capteur → analyse → communication → actionneur.

— Protocole MQTT leger et adapte aux systemes embarques.

— Signalisation claire et intuitive (LED verte / rouge / buzzer).

— Code modulaire et facilement extensible a d'autres bacs.

Annexes

A. Installation des Dependances

Python :

```
pip install opencv-python paho-mqtt
```

Bibliotheques Arduino (ESP32) :

— `WiFi` — incluse dans ESP32 Arduino Core

— `PubSubClient` by Nick O'Leary

B. Configuration du Broker MQTT

Parametre	Valeur
Broker	`broker.hivemq.com`
Port	`1883`
Client ID	`ESP32_BIN_1`
Topic statut	`city/bin/1/status`
Topic alerte	`city/bin/1/alert`

C. Lien Simulation Wokwi

Le projet est accessible sur : https://wokwi.com/projects/460228988277442561